

Week 15: Final Project: Database System Design & Implementation

Britney Sipkema

Davenport University

CISP 309 – Database Structures

Dr. Daniel Parrell

May 2, 2026

Table of Contents

INTRODUCTION	4
ENTITY RELATIONSHIP DIAGRAM (ERD)	4
DATA DICTIONARY	7
TABLE 1: REF_VEHICLE_TYPES	7
TABLE 2: REF_TICKET_STATUS.....	8
TABLE 3: VIOLATORS	8
TABLE 4: VEHICLES	9
TABLE 5: VIOLATOR_VEHICLES	10
TABLE 6: OFFICERS	11
TABLE 7: VIOLATIONS	12
TABLE 8: TICKETS	13
DATABASE STRUCTURE (DDL)	14
TABLE CREATION VERIFICATION	15
DATA MANIPULATION (DML).....	16
INSERT DATA: REF_TICKET_STATUS	16
INSERT DATA: REF_VEHICLE_TYPES.....	17
INSERT DATA: OFFICERS	18
INSERT DATA: VEHICLES	19

INSERT DATA: VIOLATORS.....	20
INSERT DATA: VIOLATOR_VEHICLES.....	21
INSERT DATA: VIOLATIONS.....	22
INSERT DATA: TICKETS.....	23
INDEXES	24
 <u>AUDIT AND INTEGRITY</u>	 <u>25</u>
 AUDIT TABLES	 25
TRIGGERS	26
TRIGGERS	27
VALIDATION FUNCTIONS	28
FUNCTION TESTING AND OUTPUT	28
STORED PROCEDURES (1)	29
STORED PROCEDURES (2)	30
STORED PROCEDURES (3)	31
 <u>REPORTING</u>	 <u>32</u>
 DETAILED REPORT	 32
SUMMARY REPORT.....	33

Introduction

This project demonstrates the design and implementation of a relational database system for managing traffic violations and ticketing. The system was developed using MySQL and includes multiple related tables such as violators, vehicles, officers, violations, and tickets. Each table is connected through primary and foreign key relationships to ensure consistency and accuracy.

In addition to basic table creation, the database incorporates several advanced features to improve functionality and maintain data integrity. These include indexes for performance optimization, triggers for tracking changes, functions for data validation, and stored procedures to simplify and control data entry. Together, these components create a structured and efficient system for storing and managing traffic-related data.

The project also includes reporting capabilities to demonstrate how data can be retrieved and analyzed. A detailed report combines information from multiple tables, while a summary report provides an overview of ticket statuses and totals. Overall, this project highlights how relational database concepts can be applied to build a complete and functional system.

Entity Relationship Diagram (ERD)

The following diagrams show the structure of the database used in this project. The first diagram represents the original data model obtained from the provided source, while the second diagram shows the customized Entity Relationship Diagram (ERD) developed in MySQL Workbench. The updated design refines table structures, establishes clear primary and foreign key

relationships, and ensures referential integrity across all entities. This transformation demonstrates how an initial concept can be translated into a fully functional relational database design.

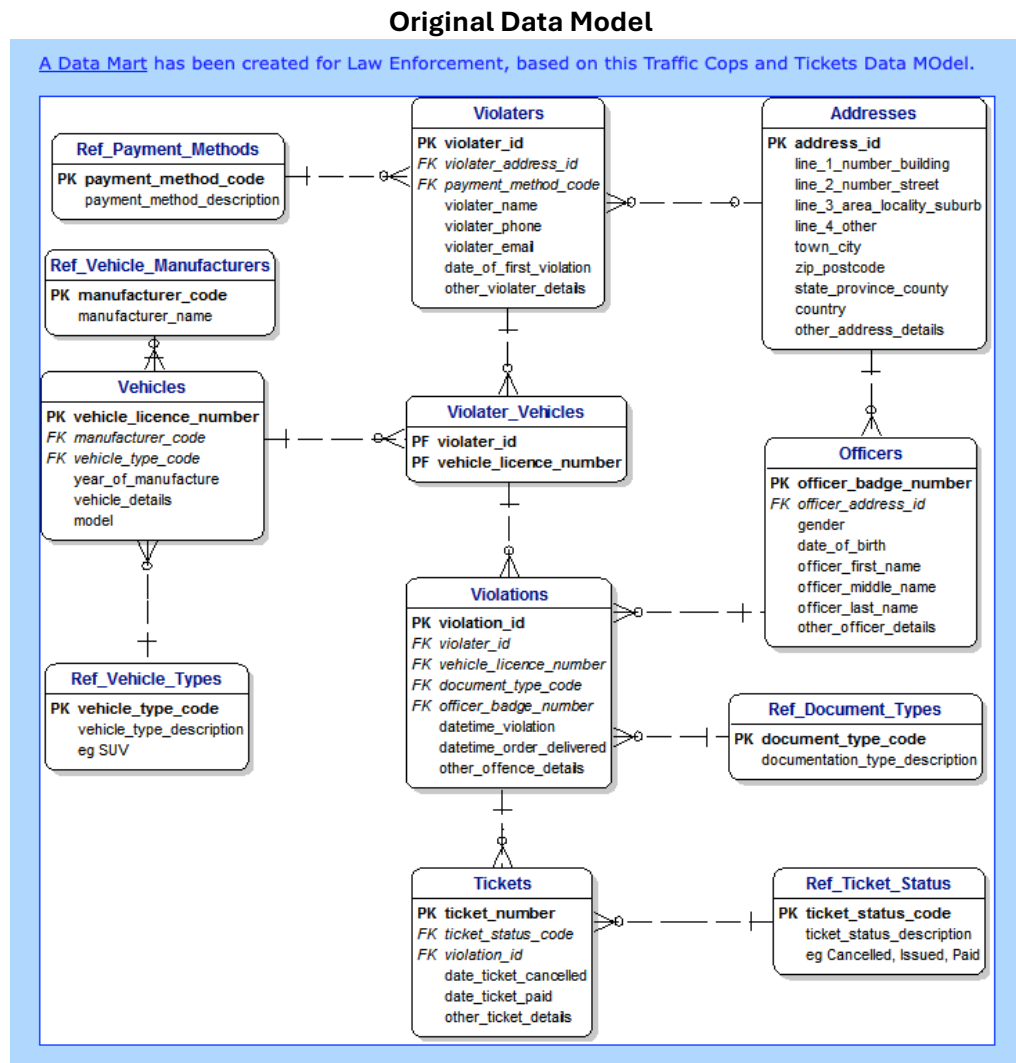


Figure 1: Source data model used as design reference

Implemented ERD in MySQL Workbench

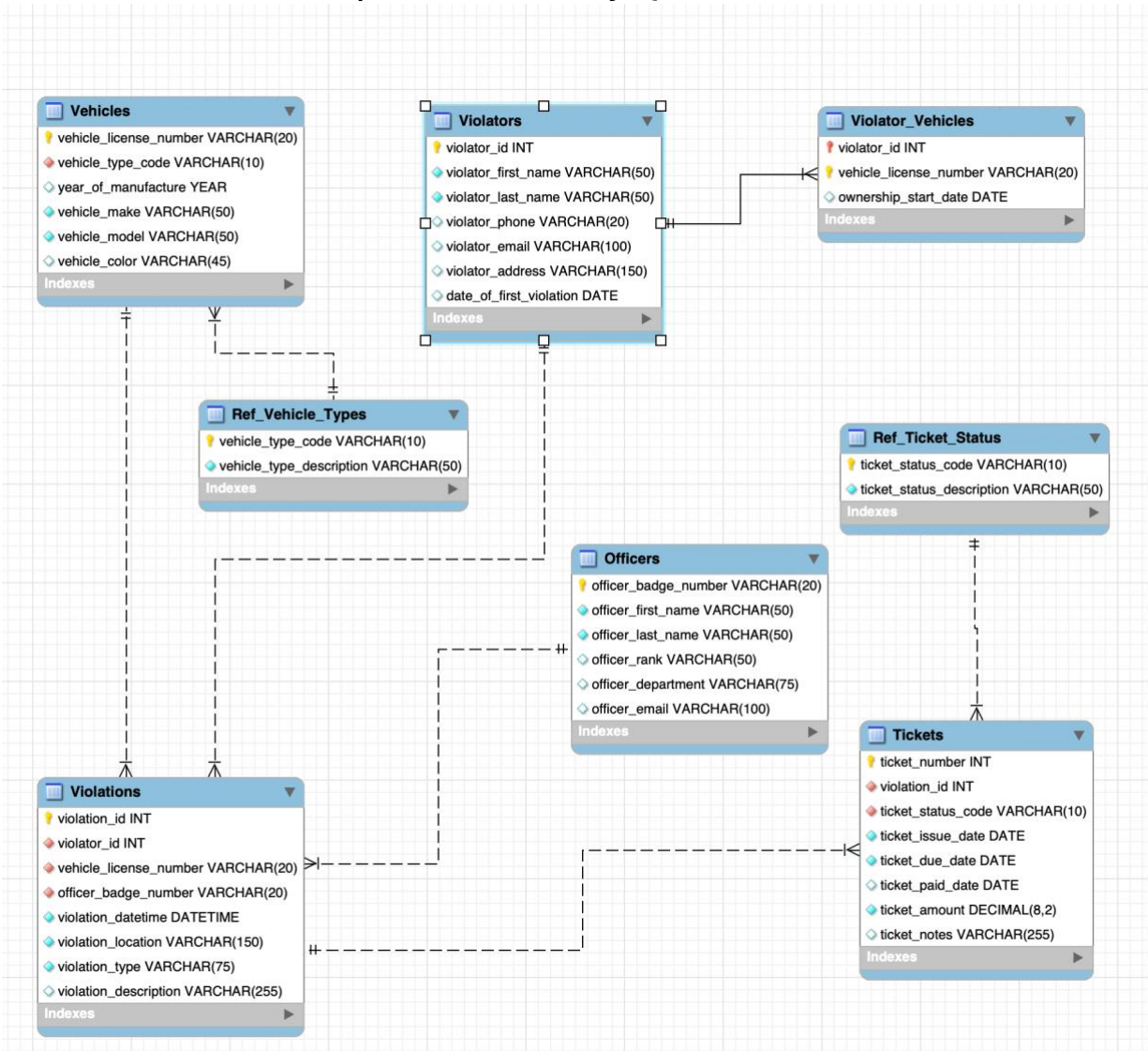


Figure 2: Final ERD with defined relationships and keys

Data Dictionary

The data dictionary provides a detailed description of each table and its columns within the database. It includes information such as data types, field lengths, key designations, and descriptions of each column. This section ensures clarity in how data is structured and helps maintain consistency and understanding throughout the database design.

Table 1: Ref_Vehicle_Types

Column Name	Data Type	Length	Nullable	Key Type	Description
vehicle_type_code	VARCHAR	10	No	Primary Key	Unique code used to identify each vehicle type ~ such as SUV, truck, sedan, or motorcycle.
vehicle_type_description	VARCHAR	50	No	None	Description of the vehicle type connected to the vehicle type code.

The Ref_Vehicle_Types table is a lookup table used to store standardized vehicle type codes and their corresponding descriptions. It helps ensure consistency across the database by limiting vehicle types to predefined values. This table is referenced by the Vehicles table through a foreign key relationship, allowing each vehicle to be categorized accurately.

Table 2: Ref_Ticket_Status

Column Name	Data Type	Length	Nullable	Key Type	Description
ticket_status_code	VARCHAR	10	No	Primary Key	Unique code used to identify the current status of a ticket ~ such as issued, paid, cancelled, or overdue.
ticket_status_description	VARCHAR	50	No	None	Description of the ticket associated with each status code.

The Ref_Ticket_Status table is a lookup table used to store predefined ticket status codes and their corresponding descriptions. It ensures consistency by restricting ticket statuses to a controlled set of values such as issued, paid, cancelled, or overdue. This table is referenced by the Tickets table through a foreign key relationship, allowing each ticket to be assigned a valid and standardized status.

Table 3: Violators

Column Name	Data Type	Length	Nullable	Key Type	Description
violator_id	INT	N/A	No	Primary Key	Unique identifier assigned to each violator in the system.
violator_first_name	VARCHAR	50	No	None	First name of violator.
violator_last_name	VARCHAR	50	No	None	Last name of violator.
violator_phone	VARCHAR	20	Yes	None	Contact phone number for violator.
violator_email	VARCHAR	100	Yes	None	Email address for violator.
violator_address	VARCHAR	150	Yes	None	Mailing or residential address of the violator.
date_of_first_violation	DATE	N/A	Yes	None	Date when the violator committed their first

					recorded violation in the system.
--	--	--	--	--	-----------------------------------

The Violators table stores information about individuals who have committed traffic violations. Each violator is uniquely identified by a primary key and includes personal and contact details such as name, phone number, email, and address. This table serves as a central entity in the database and is referenced by other tables, such as Violations and Violator_Vehicles, to associate individuals with specific incidents and vehicle ownership records.

Table 4: Vehicles

Column Name	Data Type	Length	Nullable	Key Type	Description
vehicle_license_number	VARCHAR	20	No	Primary Key	Unique license plate number used to identify the vehicle.
vehicle_type_code	VARCHAR	10	No	Foreign Key	Code that identifies the type of the vehicle and links to the Ref_Vehicle_Types table
year_of_manufacture	YEAR	N/A	Yes	None	The year the vehicle was manufactured.
vehicle_make	VARCHAR	50	No	None	Manufacturer of the vehicle ~ such as Chevrolet, Ford or Dodge.
vehicle_model	VARCHAR	50	No	None	Model name of the vehicle ~ such as Impala, Explorer, or Challenger.
vehicle_color	VARCHAR	45	Yes	None	The color of the vehicle.

The Vehicles table stores information about vehicles involved in traffic violations. Each vehicle is uniquely identified by its license plate number and includes details like vehicle type,

manufacturer, model, year, and color. The vehicle type is linked to the Ref_Vehicle_Types lookup table to ensure standardized classification. This table is referenced by other tables, such as Violations and Violator_Vehicles, to associate vehicles with specific incidents and ownership.

Table 5: Violator_Vehicles

Column Name	Data Type	Length	Nullable	Key Type	Description
violator_id	INT	N/A	No	Primary Key / Foreign Key	Identifies the violator connected to a vehicle and links it to the Violators table.
vehicle_license_number	VARCHAR	20	No	Primary Key / Foreign Key	Identifies the vehicle connected to a violator and references the Vehicles table.
ownership_start_date	DATE	N/A	Yes	None	Date when the violator became associated with or began owning the vehicle.

The Violator_Vehicles table is a junction table used to represent the many-to-many relationship between violators and vehicles. It links individuals to the vehicles they own or are associated with using a composite primary key. This structure allows a violator to be associated with multiple vehicles and a vehicle to be associated with multiple violators over time. The table also includes ownership start date to track when the relationship began.

Table 6: Officers

Column Name	Data Type	Length	Nullable	Key Type	Description
officer_badge_number	VARCHAR	20	No	Primary Key	Unique badge number used to identify each officer.
officer_first_name	VARCHAR	50	No	None	First name of the officer.
officer_last_name	VARCHAR	50	No	None	Last name of the officer.
officer_rank	VARCHAR	50	Yes	None	Rank or title of the officer ~ such as Officer, Sergeant, or Lieutenant.
officer_department	VARCHAR	75	Yes	None	Department or unit the officer is assigned to.
officer_email	VARCHAR	100	Yes	None	Email address of the officer.

The Officers table stores information about law enforcement personnel responsible for issuing traffic violations. Each officer is uniquely identified by a badge number and includes personal and organizational details such as name, rank, department, and contact information. This table is referenced by the Violations table to associate each recorded violation with the officer who issued it.

Table 7: Violations

Column Name	Data Type	Length	Nullable	Key Type	Description
violation_id	INT	N/A	No	Primary Key	Unique identifier assigned to each recorded violation.
violator_id	INT	N/A	No	Foreign Key	Identifies the violator involved in the violation and references the Violators table.
vehicle_license_number	VARCHAR	20	No	Foreign Key	Identifies the vehicle involved in the violation and links to the Vehicles table.
officer_badge_number	VARCHAR	20	No	Foreign Key	Identifies the officer who recorded or issued the violation and references the Officers table.
violation_datetime	DATETIME	N/A	No	None	Date and time when the violation occurred.
violation_location	VARCHAR	150	No	None	Location where the violation occurred.
violation_type	VARCHAR	75	No	None	Type of violation committed ~ such as speeding or at-fault accidents.
violation_description	VARCHAR	255	Yes	None	Additional details or notes about the violation.

The Violations table stores detailed information about each recorded traffic violation. Each violation is uniquely identified by a primary key and is associated with a specific violator, vehicle, and officer through foreign key relationships. This table captures key details such as the date and time, location, type, and description of the violation. It serves as a central table in the database and is referenced by the Tickets table to link violations to issued tickets.

Table 8: Tickets

Column Name	Data Type	Length	Nullable	Key Type	Description
ticket_number	INT	N/A	No	Primary Key	Unique number assigned to each ticket in the system.
violation_id	INT	N/A	No	Foreign Key	Identifies the violation connected to the ticket and references the Violations table.
ticket_status_code	VARCHAR	10	No	Foreign Key	Identifies the current status of the ticket and links to the Ref_Ticket_Status table.
ticket_issue_date	DATE	N/A	No	None	Date the ticket was issued.
ticket_due_date	DATE	N/A	No	None	Date the ticket payment is due.
ticket_paid_date	DATE	N/A	Yes	None	Date the ticket was paid ~ if applicable.
ticket_amount	DECIMAL	8, 2	No	None	Dollar amount assigned to the ticket.
ticket_notes	VARCHAR	255	Yes	None	Additional notes or comments related to the ticket.

The Tickets table stores information related to citations issued for traffic violations. Each ticket is uniquely identified by a primary key and is linked to a specific violation through a foreign key relationship. The table includes details such as ticket status, issue date, due date, payment date, and monetary amount. It also references the Ref_Ticket_Status lookup table to ensure consistent classification of ticket statuses. This table represents the final stage in the workflow, connecting violations to their associated penalties and outcomes.

Database Structure (DDL)

The database structure was created using Data Definition Language (DDL) statements in MySQL. These statements created each table, assigned primary keys, established foreign key relationships, and used AUTO_INCREMENT fields where appropriate for generated identifiers.

<pre>-- This table stores valid vehicle type values CREATE TABLE Ref_Vehicle_Types(vehicle_type_code VARCHAR(10) PRIMARY KEY, vehicle_type_description VARCHAR(50) NOT NULL);</pre>	<pre>-- This table stores valid ticket status values CREATE TABLE Ref_Ticket_Status(ticket_status_code VARCHAR(10) PRIMARY KEY, ticket_status_description VARCHAR(50) NOT NULL);</pre>
<pre>-- VIOLATORS TABLE: -- This table stores people who receive violations CREATE TABLE Violators(violator_id INT AUTO_INCREMENT PRIMARY KEY, violator_first_name VARCHAR(50) NOT NULL, violator_last_name VARCHAR(50) NOT NULL, violator_phone VARCHAR(20), violator_email VARCHAR(100), violator_address VARCHAR(150), date_of_first_violation DATE);</pre>	<pre>CREATE TABLE Vehicles(vehicle_license_number VARCHAR(20) PRIMARY KEY, vehicle_type_code VARCHAR(10) NOT NULL, year_of_manufacture YEAR, vehicle_make VARCHAR(50) NOT NULL, vehicle_model VARCHAR(50) NOT NULL, vehicle_color VARCHAR(45), CONSTRAINT fk_vehicle_type_code FOREIGN KEY(vehicle_type_code) REFERENCES Ref_Vehicle_Types(vehicle_type_code));</pre>
<pre>CREATE TABLE Officers(officer_badge_number VARCHAR(20) PRIMARY KEY, officer_first_name VARCHAR(50) NOT NULL, officer_last_name VARCHAR(50) NOT NULL, officer_rank VARCHAR(50), officer_department VARCHAR(75), officer_email VARCHAR(100));</pre>	<pre>CREATE TABLE Violator_Vehicles(violator_id INT NOT NULL, vehicle_license_number VARCHAR(20) NOT NULL, ownership_start_date DATE, PRIMARY KEY(violator_id, vehicle_license_number), CONSTRAINT fk_vv_violator_id FOREIGN KEY(violator_id) REFERENCES Violators(violator_id), CONSTRAINT fk_vv_vehicle_license FOREIGN KEY(vehicle_license_number) REFERENCES Vehicles(vehicle_license_number));</pre>

```

CREATE TABLE Violations(
    violation_id          INT AUTO_INCREMENT PRIMARY KEY,
    violator_id           INT NOT NULL,
    vehicle_license_number VARCHAR(20) NOT NULL,
    officer_badge_number  VARCHAR(20) NOT NULL,
    violation_datetime     DATETIME NOT NULL,
    violation_location     VARCHAR(150) NOT NULL,
    violation_type         VARCHAR(75) NOT NULL,
    violation_description  VARCHAR(255),

    CONSTRAINT fk_violations_violator
        FOREIGN KEY (violator_id)
        REFERENCES Violators(violator_id),

    CONSTRAINT fk_violations_vehicle
        FOREIGN KEY (vehicle_license_number)
        REFERENCES Vehicles(vehicle_license_number),

    CONSTRAINT fk_violations_officer
        FOREIGN KEY (officer_badge_number)
        REFERENCES Officers(officer_badge_number)
);

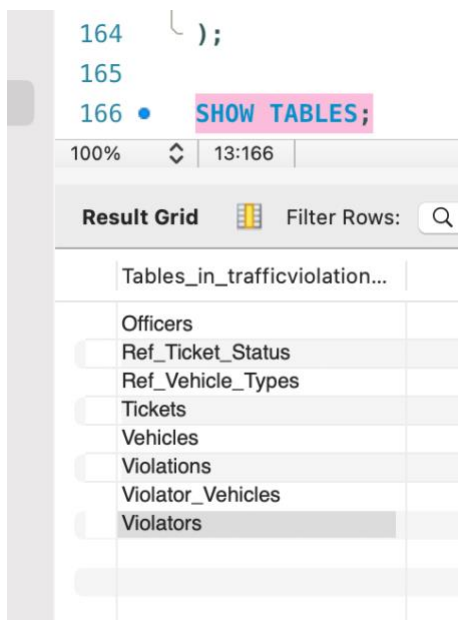
CREATE TABLE Tickets(
    ticket_number          INT AUTO_INCREMENT PRIMARY KEY,
    violation_id           INT NOT NULL,
    ticket_status_code     VARCHAR(10) NOT NULL,
    ticket_issue_date      DATE NOT NULL,
    ticket_due_date        DATE NOT NULL,
    ticket_paid_date       DATE,
    ticket_amount          DECIMAL(8,2) NOT NULL,
    ticket_notes           VARCHAR(255),

    CONSTRAINT fk_tickets_violation
        FOREIGN KEY (violation_id)
        REFERENCES Violations(violation_id),

    CONSTRAINT fk_ticket_status
        FOREIGN KEY (ticket_status_code)
        REFERENCES Ref_Ticket_Status(ticket_status_code)
);

```

Table Creation Verification



The SHOW TABLES command was used to verify that all tables were successfully created within the database. The output confirms the presence of all expected tables, including lookup tables, core entity tables, and relationship tables. This step serves as a validation check to ensure that the database structure was implemented correctly before proceeding to data insertion and further operations.

Data Manipulation (DML)

Data Manipulation Language (DML) statements were used to insert sample data into each table within the database. A minimum of 15 records were added to each primary table to ensure sufficient data for testing relationships, queries, and reporting. The insertion process followed the correct order of foreign key dependencies, starting with lookup tables and progressing to core and transactional tables.

Insert Data: Ref_Ticket_Status

<pre> • INSERT INTO Ref_Ticket_Status (ticket_status_code, ticket_status_description) VALUES ('ISSUED', 'Ticket has been issued but not yet paid'), ('PAID', 'Ticket has been paid'), ('OVERDUE', 'Ticket payment is past the due date'), ('CANCELLED', 'Ticket has been cancelled'), ('APPEAL', 'Ticket is under appeal'), ('DISMISSED', 'Ticket has been dismissed'), ('VOID', 'Ticket was entered in error and voided'); • SELECT * FROM Ref_Vehicle_Types; • SELECT * FROM Ref_Ticket_Status; </pre>		
<i>INSERT statements for Ref_Ticket_Status table</i>		
ticket_status_co...	ticket_status_description	
APPEAL	Ticket is under appeal	
CANCELLED	Ticket has been cancelled	
DISMISSED	Ticket has been dismissed	
ISSUED	Ticket has been issued but not yet paid	
OVERDUE	Ticket payment is past the due date	
PAID	Ticket has been paid	
VOID	Ticket was entered in error and voided	
HULL	HULL	
<i>Ref_Ticket_Status table populated with sample data</i>		

The Ref_Ticket_Status table was populated with predefined ticket status values to ensure consistent classification across the database. These values serve as a lookup reference for the Tickets table, allowing each ticket to be assigned a valid and standardized status.

Insert Data: Ref_Vehicle_Types

```
INSERT INTO Ref_Vehicle_Types
(vehicle_type_code, vehicle_type_description)
VALUES
('SEDAN', 'Sedan'),
('SUV', 'Sport Utility Vehicle'),
('TRUCK', 'Pickup Truck'),
('VAN', 'Van'),
('MOTOR', 'Motorcycle'),
('BUS', 'Bus'),
('COUPE', 'Coupe'),
('CONV', 'Convertible'),
('WAGON', 'Station Wagon'),
('SEMI', 'Semi Truck'),
('TRAILER', 'Trailer'),
('TAXI', 'Taxi'),
('EV', 'Electric Vehicle'),
('HYBRID', 'Hybrid Vehicle'),
('OTHER', 'Other Vehcile Types');
```

INSERT statements for Ref_Vehicle_Types table

vehicle_type_co...	vehicle_type_descript...
BUS	Bus
CONV	Convertible
COUPE	Coupe
EV	Electric Vehicle
HYBRID	Hybrid Vehicle
MOTOR	Motorcycle
OTHER	Other Vehcile Types
SEDAN	Sedan
SEMI	Semi Truck
SUV	Sport Utility Vehicle
TAXI	Taxi
TRAILER	Trailer
TRUCK	Pickup Truck
VAN	Van
WAGON	Station Wagon
NULL	NULL

Ref_Vehicle_Types table populated with sample data

The Ref_Vehicle_Types table was populated with predefined categories to ensure consistent classification across the database. These values act as a lookup reference for the Vehicles table, allowing each vehicle to be assigned a valid and standardized type. This structure helps maintain data integrity by preventing inconsistent or invalid type entries.

Insert Data: Officers

249	-- =====
250	-- SECTION 7: INSERT OFFICERS DATA
251	-- =====
252	• INSERT INTO Officers
253	(officer_badge_number,
254	officer_first_name,
255	officer_last_name,
256	officer_rank,
257	officer_department,
258	officer_email)
259	
260	VALUES
261	('B1001', 'James', 'Anderson', 'Officer', 'Traffic Division', 'james.anderson@finalproject.com'),
262	('B1002', 'Maria', 'Lopez', 'Sergeant', 'Traffic Division', 'maria.lopez@finalproject.com'),
263	('B1003', 'Robert', 'Taylor', 'Officer', 'Patrol', 'robert.taylor@finalproject.com'),
264	('B1004', 'Linda', 'Harris', 'Lieutenant', 'Traffic Division', 'linda.harris@finalproject.com'),
265	('B1005', 'William', 'Clark', 'Officer', 'Patrol', 'william.clark@finalproject.com'),
266	('B1006', 'Patricia', 'Lewis', 'Sergeant', 'Patrol', 'patricia.lewis@finalproject.com'),
267	('B1007', 'Charles', 'Walker', 'Officer', 'Traffic Division', 'charles.walker@finalproject.com'),
268	('B1008', 'Barbara', 'Hall', 'Officer', 'Traffic Division', 'barbara.hall@finalproject.com'),
269	('B1009', 'Thomas', 'Allen', 'Captain', 'Traffic Division', 'thomas.allen@finalproject.com'),
270	('B1010', 'Jennifer', 'Young', 'Officer', 'Patrol', 'jennifer.young@finalproject.com'),
271	('B1011', 'Christopher', 'King', 'Officer', 'Traffic Division', 'christopher.king@finalproject.com'),
272	('B1012', 'Susan', 'Wright', 'Sergeant', 'Traffic Division', 'susan.wright@finalproject.com'),
273	('B1013', 'Daniel', 'Scott', 'Officer', 'Patrol', 'daniel.scott@finalproject.com'),
274	('B1014', 'Karen', 'Green', 'Lieutenant', 'Traffic Division', 'karen.green@finalproject.com'),
275	('B1015', 'Matthew', 'Baker', 'Officer', 'Patrol', 'matthew.baker@finalproject.com');
276	

INSERT statements for Officers table					
officer_badge_num...	officer_first_na...	officer_last_na...	officer_ra...	officer_departm...	officer_email
B1001	James	Anderson	Officer	Traffic Division	james.anderson@finalproject.com
B1002	Maria	Lopez	Sergeant	Traffic Division	maria.lopez@finalproject.com
B1003	Robert	Taylor	Officer	Patrol	robert.taylor@finalproject.com
B1004	Linda	Harris	Lieutenant	Traffic Division	linda.harris@finalproject.com
B1005	William	Clark	Officer	Patrol	william.clark@finalproject.com
B1006	Patricia	Lewis	Sergeant	Patrol	patricia.lewis@finalproject.com
B1007	Charles	Walker	Officer	Traffic Division	charles.walker@finalproject.com
B1008	Barbara	Hall	Officer	Traffic Division	barbara.hall@finalproject.com
B1009	Thomas	Allen	Captain	Traffic Division	thomas.allen@finalproject.com
B1010	Jennifer	Young	Officer	Patrol	jennifer.young@finalproject.com
B1011	Christopher	King	Officer	Traffic Division	christopher.king@finalproject.com
B1012	Susan	Wright	Sergeant	Traffic Division	susan.wright@finalproject.com
B1013	Daniel	Scott	Officer	Patrol	daniel.scott@finalproject.com
B1014	Karen	Green	Lieutenant	Traffic Division	karen.green@finalproject.com
B1015	Matthew	Baker	Officer	Patrol	matthew.baker@finalproject.com
NULL	NULL	NULL	NULL	NULL	NULL

Officers table populated with sample data					
---	--	--	--	--	--

The Officers table was populated with sample data representing law enforcement personnel responsible for issuing traffic violations. Each record includes identifying and organizational information such as badge number, name, rank, department, and contact email. This data is used to establish relationships with the Violations table, ensuring that each recorded violation can be associated with a specific officer.

Insert Data: Vehicles

<pre> INSERT INTO Vehicles (vehicle_license_number, vehicle_type_code, year_of_manufacture, vehicle_make, vehicle_model, vehicle_color) VALUES ('ABC1234', 'SEDAN', 2018, 'Toyota', 'Camry', 'Black'), ('XYZ5678', 'SUV', 2020, 'Honda', 'CR-V', 'White'), ('LMN8910', 'TRUCK', 2017, 'Ford', 'F-150', 'Blue'), ('QRS2345', 'VAN', 2019, 'Dodge', 'Caravan', 'Silver'), ('TUV6789', 'MOTOR', 2021, 'Yamaha', 'MT-07', 'Red'), ('JKL3456', 'COUPE', 2024, 'BMW', 'M4', 'Black'), ('DEF7890', 'CONV', 2025, 'Chevrolet', 'Camaro', 'Yellow'), ('GHI1122', 'WAGON', 2018, 'Subaru', 'Outback', 'Green'), ('JKM3344', 'SEMI', 2014, 'Freightliner', 'Cascadia', 'White'), ('NOP5566', 'TRAILER', 2013, 'Utility', 'Trailer', 'Gray'), ('RST7788', 'TAXI', 2019, 'Toyota', 'Prius', 'Yellow'), ('UVW9900', 'EV', 2022, 'Tesla', 'Model 3', 'White'), ('AAA1111', 'HYBRID', 2023, 'Toyota', 'Corolla Hybrid', 'Blue'), ('BBB2222', 'SUV', 2020, 'Jeep', 'Grand Cherokee', 'Black'), ('CCC3333', 'SEDAN', 2017, 'Nissan', 'Altima', 'Silver'); </pre>						
INSERT statements for Vehicles table						
vehicle_license_num...	vehicle_type_co...	year_of_manufact...	vehicle_make	vehicle_model	vehicle_color	
AAA1111	HYBRID	2023	Toyota	Corolla Hybrid	Blue	
ABC1234	SEDAN	2018	Toyota	Camry	Black	
BBB2222	SUV	2020	Jeep	Grand Cherokee	Black	
CCC3333	SEDAN	2017	Nissan	Altima	Silver	
DEF7890	CONV	2025	Chevrolet	Camaro	Yellow	
GHI1122	WAGON	2018	Subaru	Outback	Green	
JKL3456	COUPE	2024	BMW	M4	Black	
JKM3344	SEMI	2014	Freightliner	Cascadia	White	
LMN8910	TRUCK	2017	Ford	F-150	Blue	
NOP5566	TRAILER	2013	Utility	Trailer	Gray	
QRS2345	VAN	2019	Dodge	Caravan	Silver	
RST7788	TAXI	2019	Toyota	Prius	Yellow	
TUV6789	MOTOR	2021	Yamaha	MT-07	Red	
UVW9900	EV	2022	Tesla	Model 3	White	
XYZ5678	SUV	2020	Honda	CR-V	White	
NULL	NULL	NULL	NULL	NULL	NULL	
Vehicles table populated with sample data						

The Vehicles table was populated with sample data representing a variety of vehicle types, manufacturers, models, and production years. Each vehicle is assigned a valid vehicle type code that references the Ref_Vehicle_Types lookup table, ensuring consistent classification. This data supports relationships with the Violations and Violator_Vehicles tables, allowing vehicles to be linked to both traffic incidents and ownership records.

Insert Data: Violators

```

INSERT INTO Violators
(violator_first_name,
 violator_last_name,
 violator_phone,
 violator_email,
 violator_address,
 date_of_first_violation)

VALUES
('John', 'Smith', '616-555-1001', 'john.smith@finalproject.com', '123 Maple St, Grand Rapids, MI', '2023-01-15'),
('Emily', 'Johnson', '616-555-1002', 'emily.johnson@finalproject.com', '456 Oak Ave, Grand Rapids, MI', '2023-02-10'),
('Michael', 'Williams', '616-555-1003', 'michael.williams@finalproject.com', '789 Pine Rd, Wyoming, MI', '2023-03-05'),
('Sarah', 'Brown', '616-555-1004', 'sarah.brown@finalproject.com', '321 Birch St, Kentwood, MI', '2023-01-25'),
('David', 'Jones', '616-555-1005', 'david.jones@finalproject.com', '654 Cedar Ln, Grandville, MI', '2023-04-12'),
('Jessica', 'Garcia', '616-555-1006', 'jessica.garcia@finalproject.com', '987 Elm St, Walker, MI', '2024-02-18'),
('Daniel', 'Martinez', '616-555-1007', 'daniel.martinez@finalproject.com', '159 Spruce Dr, Holland, MI', '2025-05-01'),
('Ashley', 'Rodriguez', '616-555-1008', 'ashley.rodriguez@finalproject.com', '753 Willow Ct, Zeeland, MI', '2025-03-22'),
('Matthew', 'Davis', '616-555-1009', 'matthew.davis@finalproject.com', '852 Aspen Way, Byron Center, MI', '2023-01-30'),
('Amanda', 'Lopez', '616-555-1010', 'amanda.lopez@finalproject.com', '951 Cherry St, Grand Rapids, MI', '2025-04-08'),
('Joshua', 'Gonzalez', '616-555-1011', 'joshua.gonzalez@finalproject.com', '147 Walnut St, Wyoming, MI', '2026-02-27'),
('Stephanie', 'Wilson', '616-555-1012', 'stephanie.wilson@finalproject.com', '258 Poplar Ave, Kentwood, MI', '2024-03-14'),
('Andrew', 'Anderson', '616-555-1013', 'andrew.anderson@finalproject.com', '369 Hickory Rd, Grandville, MI', '2023-05-10'),
('Nicole', 'Thomas', '616-555-1014', 'nicole.thomas@finalproject.com', '741 Redwood Dr, Walker, MI', '2025-04-20'),
('Ryan', 'Taylor', '616-555-1015', 'ryan.taylor@finalproject.com', '852 Magnolia Ln, Holland, MI', '2025-11-23');

```

INSERT statements for Violators table

	violator_id	violator_first_na...	violator_last_na...	violator_pho...	violator_email	violator_address	date_of_first_violat
	1	John	Smith	616-555-1001	john.smith@finalproject.com	123 Maple St, Grand Rapids, MI	2023-01-15
	2	Emily	Johnson	616-555-1002	emily.johnson@finalproject.com	456 Oak Ave, Grand Rapids, MI	2023-02-10
	3	Michael	Williams	616-555-1003	michael.williams@finalproject.com	789 Pine Rd, Wyoming, MI	2023-03-05
	4	Sarah	Brown	616-555-1004	sarah.brown@finalproject.com	321 Birch St, Kentwood, MI	2023-01-25
	5	David	Jones	616-555-1005	david.jones@finalproject.com	654 Cedar Ln, Grandville, MI	2023-04-12
	6	Jessica	Garcia	616-555-1006	jessica.garcia@finalproject.com	987 Elm St, Walker, MI	2024-02-18
	7	Daniel	Martinez	616-555-1007	daniel.martinez@finalproject.com	159 Spruce Dr, Holland, MI	2025-05-01
	8	Ashley	Rodriguez	616-555-1008	ashley.rodriguez@finalproject.com	753 Willow Ct, Zeeland, MI	2025-03-22
	9	Matthew	Davis	616-555-1009	matthew.davis@finalproject.com	852 Aspen Way, Byron Center, MI	2023-01-30
	10	Amanda	Lopez	616-555-1010	amanda.lopez@finalproject.com	951 Cherry St, Grand Rapids, MI	2025-04-08
	11	Joshua	Gonzalez	616-555-1011	joshua.gonzalez@finalproject.com	147 Walnut St, Wyoming, MI	2026-02-27
	12	Stephanie	Wilson	616-555-1012	stephanie.wilson@finalproject.com	258 Poplar Ave, Kentwood, MI	2024-03-14
	13	Andrew	Anderson	616-555-1013	andrew.anderson@finalproject.com	369 Hickory Rd, Grandville, MI	2023-05-10
	14	Nicole	Thomas	616-555-1014	nicole.thomas@finalproject.com	741 Redwood Dr, Walker, MI	2025-04-20
	15	Ryan	Taylor	616-555-1015	ryan.taylor@finalproject.com	852 Magnolia Ln, Holland, MI	2025-11-23
		NULL	NULL	NULL	NULL	NULL	NULL

Violators table populated with sample data

The Violators table was populated with sample data representing individuals involved in traffic violations. Each record includes identifying and contact information such as name, phone number, email, and address, along with the date of their first recorded violation. This data supports relationships with the Violations and Violator_Vehicles tables, allowing individuals to be linked to specific incidents and associated vehicles.

Insert Data: Violator_Vehicles

<pre> INSERT INTO Violator_Vehicles (violator_id, vehicle_license_number, ownership_start_date) VALUES (1, 'ABC1234', '2022-01-10'), (2, 'XYZ5678', '2021-06-15'), (3, 'LMN8910', '2020-03-20'), (4, 'QRS2345', '2022-11-05'), (5, 'TUV6789', '2023-02-18'), (6, 'JKL3456', '2019-07-22'), (7, 'DEF7890', '2018-09-30'), (8, 'GHI1122', '2021-12-01'), (9, 'JKM3344', '2017-04-17'), (10, 'NOP5566', '2016-08-09'), (11, 'RST7788', '2020-10-14'), (12, 'UVW9900', '2023-01-25'), (13, 'AAA1111', '2022-06-06'), (14, 'BBB2222', '2021-03-11'), (15, 'CCC3333', '2019-05-27'); </pre>		
<i>INSERT statements for Violator_Vehicles table</i>		
violator_id	vehicle_license_num...	ownership_start_d...
1	ABC1234	2022-01-10
2	XYZ5678	2021-06-15
3	LMN8910	2020-03-20
4	QRS2345	2022-11-05
5	TUV6789	2023-02-18
6	JKL3456	2019-07-22
7	DEF7890	2018-09-30
8	GHI1122	2021-12-01
9	JKM3344	2017-04-17
10	NOP5566	2016-08-09
11	RST7788	2020-10-14
12	UVW9900	2023-01-25
13	AAA1111	2022-06-06
14	BBB2222	2021-03-11
15	CCC3333	2019-05-27
NULL	NULL	NULL
<i>Violator_Vehicles table populated with sample data</i>		

The Violator_Vehicles table was populated with sample data to establish relationships between violators and their associated vehicles. This table represents a junction table that resolves the many-to-many relationship between the Violators and Vehicles tables. Each record links a specific violator to a vehicle, along with the date when the ownership or association began. This structure allows multiple vehicles to be associated with a single violator and enables tracking of vehicle ownership over time.

Insert Data: Violations

```
-- SECTION 10: INSERT VIOLATIONS DATA
--=====

INSERT INTO Violations
(violator_id,
vehicle_license_number,
officer_badge_number,
violation_datetime,
violation_location,
violation_type,
violation_description)

VALUES
(1, 'ABC1234', 'B1001', '2023-06-01 08:30:00', 'Grand Rapids, MI', 'Speeding', 'Exceeded speed limit by 15 mph'),
(2, 'XYZ5678', 'B1002', '2023-06-02 09:45:00', 'Wyoming, MI', 'Red Light', 'Failed to stop at red light'),
(3, 'LMN8910', 'B1003', '2023-06-03 11:15:00', 'Kentwood, MI', 'Illegal Turn', 'Made an illegal left turn'),
(4, 'QRS2345', 'B1004', '2023-06-04 14:20:00', 'Grandville, MI', 'Speeding', 'Exceeded speed limit by 10 mph'),
(5, 'TUV6789', 'B1005', '2023-06-05 16:05:00', 'Walker, MI', 'No Helmet', 'Motorcycle rider without helmet'),
(6, 'JKL3456', 'B1006', '2023-06-06 10:10:00', 'Holland, MI', 'Expired Plates', 'Vehicle registration expired'),
(7, 'DEF7890', 'B1007', '2023-06-07 13:55:00', 'Zeeland, MI', 'Speeding', 'Exceeded speed limit by 20 mph'),
(8, 'GHI1122', 'B1008', '2023-06-08 07:40:00', 'Byron Center, MI', 'Seatbelt', 'Driver not wearing seatbelt'),
(9, 'JKM3344', 'B1009', '2023-06-09 18:25:00', 'Grand Rapids, MI', 'Overweight Load', 'Truck exceeded weight limit'),
(10, 'NOP5566', 'B1010', '2023-06-10 12:00:00', 'Wyoming, MI', 'Improper Load', 'Trailer load not secured properly'),
(11, 'RST7788', 'B1011', '2023-06-11 15:35:00', 'Kentwood, MI', 'Reckless Driving', 'Aggressive lane changes observed'),
(12, 'UVW9900', 'B1012', '2023-06-12 09:10:00', 'Grandville, MI', 'Speeding', 'Exceeded speed limit by 12 mph'),
(13, 'AAA1111', 'B1013', '2023-06-13 11:50:00', 'Walker, MI', 'Distracted', 'Using phone while driving'),
(14, 'BBB2222', 'B1014', '2023-06-14 13:30:00', 'Holland, MI', 'Tailgating', 'Following too closely'),
(15, 'CCC3333', 'B1015', '2023-06-15 17:45:00', 'Zeeland, MI', 'Speeding', 'Exceeded speed limit by 18 mph');

SELECT * FROM Violations;
```

INSERT statements for Violations table

	violation_id	violator_id	vehicle_license_num...	officer_badge_num...	violation_datetime	violation_locati...	violation_type	violation_description
	1	1	ABC1234	B1001	2023-06-01 08:30:00	Grand Rapids, MI	Speeding	Exceeded speed limit by 15 mph
	2	2	XYZ5678	B1002	2023-06-02 09:45:00	Wyoming, MI	Red Light	Failed to stop at red light
	3	3	LMN8910	B1003	2023-06-03 11:15:00	Kentwood, MI	Illegal Turn	Made an illegal left turn
	4	4	QRS2345	B1004	2023-06-04 14:20:00	Grandville, MI	Speeding	Exceeded speed limit by 10 mph
	5	5	TUV6789	B1005	2023-06-05 16:05:00	Walker, MI	No Helmet	Motorcycle rider without helmet
	6	6	JKL3456	B1006	2023-06-06 10:10:00	Holland, MI	Expired Plates	Vehicle registration expired
	7	7	DEF7890	B1007	2023-06-07 13:55:00	Zeeland, MI	Speeding	Exceeded speed limit by 20 mph
	8	8	GHI1122	B1008	2023-06-08 07:40:00	Byron Center, MI	Seatbelt	Driver not wearing seatbelt
	9	9	JKM3344	B1009	2023-06-09 18:25:00	Grand Rapids, MI	Overweight Load	Truck exceeded weight limit
	10	10	NOP5566	B1010	2023-06-10 12:00:00	Wyoming, MI	Improper Load	Trailer load not secured properly
	11	11	RST7788	B1011	2023-06-11 15:35:00	Kentwood, MI	Reckless Driving	Aggressive lane changes obse...
	12	12	UVW9900	B1012	2023-06-12 09:10:00	Grandville, MI	Speeding	Exceeded speed limit by 12 mph
	13	13	AAA1111	B1013	2023-06-13 11:50:00	Walker, MI	Distracted	Using phone while driving
	14	14	BBB2222	B1014	2023-06-14 13:30:00	Holland, MI	Tailgating	Following too closely
	15	15	CCC3333	B1015	2023-06-15 17:45:00	Zeeland, MI	Speeding	Exceeded speed limit by 18 mph
	HULL	HULL	HULL	HULL	HULL	HULL	HULL	HULL

Violations table populated with sample data

The Violations table was populated with sample data representing recorded traffic violations. Each record links a violator, vehicle, and officer through foreign key relationships, ensuring referential integrity across the database. The data includes detailed information such as the date and time of the violation, location, type of violation, and a descriptive summary. This table serves as the central component of the database, connecting individuals, vehicles, and enforcement actions.

Insert Data: Tickets

The tickets table was populated with sample data representing citations issued for recorded traffic violations. Each ticket is linked to a specific violation through a foreign key relationship and includes details such as ticket status, issue date, due date, payment date, and monetary amount. The ticket stat is validated through the Ref_Ticket_Status lookup table to ensure consistency. This table represents the final stage in the process, capturing the outcome and financial impact of each violation.

Indexes

Indexes were created to improve query performance by allowing faster data retrieval on frequently searched and joined columns. These indexes optimize operations such as filtering, sorting, and joining across tables, particularly on key attributes used in reporting and relational queries.

```
-- =====
-- SECTION 12: CREATE INDEXES
-- ** Indexes improve search and join
--    performance on commonly used fields.
-- =====

CREATE INDEX idx_violators_last_name
ON Violators (violator_last_name);

CREATE INDEX idx_vehicles_type
ON Vehicles (vehicle_type_code);

CREATE INDEX idx_violations_datetime
ON Violations (violation_datetime);

CREATE INDEX idx_violations_type
ON Violations (violation_type);

CREATE INDEX idx_tickets_status
ON Tickets (ticket_status_code);

CREATE INDEX idx_tickets_due_date
ON Tickets (ticket_due_date);

SHOW INDEX FROM Violations;
SHOW INDEX FROM Tickets;
```

CREATE INDEX statements to optimize query performance

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_ty...	Comment	Index_comment	Visible	Expression
Violations	0	PRIMARY	1	violation_id	A	15				BTREE			YES	
Violations	1	fk_violations_violator	1	violator_id	A	15				BTREE			YES	
Violations	1	fk_violations_vehicle	1	vehicle_license_number	A	15				BTREE			YES	
Violations	1	fk_violations_officer	1	officer_badge_number	A	15				BTREE			YES	
Violations	1	idx_violations_datetime	1	violation_datetime	A	15				BTREE			YES	
Violations	1	idx_violations_type	1	violation_type	A	11				BTREE			YES	

Sample index output confirming index creation on the Violations table

Indexes were created to improve query performance by allowing faster data retrieval on frequently searched and joined columns. While primary and foreign keys are automatically indexed by MySQL, additional indexes were implemented on non-key attributes that are commonly used in filtering and reporting operations. For example, indexes on violator_last_name and vehicle_type_code support efficient lookup and categorization, while indexes on

violation_datetime and violation_type improve performance for time-based and analytical queries.

Similarly, indexes on ticket_status_code and ticket_due_date enhance the efficiency of ticket tracking and status-based reporting. The sample output shown confirms that the indexes were successfully created and are actively available for query optimization.

Audit and Integrity

The Audit and Integrity components were implemented to ensure data accuracy, enforce business rules, and track changes within the database. This section includes the creation of audit tables, triggers to automatically capture changes, validation functions, and stored procedures designed to maintain referential integrity and support-controlled data operations.

Audit Tables

```

421  -- =====
422  -- SECTION 13: CREATE AUDIT TABLES
423  -- =====
424
425  -- This audit table tracks changes made to ticket records
426
427  • CREATE TABLE Ticket_Audit(
428      audit_id          INT AUTO_INCREMENT PRIMARY KEY,
429      ticket_number     INT,
430      old_status_code   VARCHAR(10),
431      new_status_code   VARCHAR(10),
432      old_ticket_amount DECIMAL(8,2),
433      new_ticket_amount DECIMAL(8,2),
434      action_type       VARCHAR(20),
435      changed_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
436  );
437
438  -- This audit table tracks changes made to violation records.
439
440  • CREATE TABLE Violation_Audit(
441      audit_id          INT AUTO_INCREMENT PRIMARY KEY,
442      violation_id       INT,
443      old_violation_type VARCHAR(75),
444      new_violation_type VARCHAR(75),
445      old_violation_location VARCHAR(150),
446      new_violation_location VARCHAR(150),
447      action_type       VARCHAR(20),
448      changed_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
449  );
450
451  • SHOW TABLES;

```

CREATE TABLE statements for audit tables

Tables_in_trafficviolation...	
Officers	
Ref_Ticket_Status	
Ref_Vehicle_Types	
Ticket_Audit	
Tickets	
Vehicles	
Violation_Audit	
Violations	
Violator_Vehicles	
Violators	

Audit tables successfully created in the database

Two audit tables were created to track changes made to key data within the database. The Ticket_Audit table records updates to ticket status and amounts, while the Violation_Audit table captures changes to violation type and location. Each audit record stores both the original and updated values, along with the action performed and a timestamp. This ensures that all important changes are logged and can be reviewed for accuracy and accountability.

Triggers

```

452  -- =====
453  -- SECTION 14: TRIGGER FOR TICKETS (UPDATE)
454  -- =====
455
456  DELIMITER $$
457
458  • CREATE TRIGGER trg_ticket_update
459  AFTER UPDATE ON Tickets
460  FOR EACH ROW
461  BEGIN
462      INSERT INTO Ticket_Audit
463      (
464          ticket_number,
465          old_status_code,
466          new_status_code,
467          old_ticket_amount,
468          new_ticket_amount,
469          action_type
470      )
471      VALUES
472      (
473          OLD.ticket_number,
474          OLD.ticket_status_code,
475          NEW.ticket_status_code,
476          OLD.ticket_amount,
477          NEW.ticket_amount,
478          'UPDATE'
479      );
480  END$$
481
482  DELIMITER ;

```

Trigger to capture updates to the Tickets table

audit_id	ticket_num...	old_status_co...	new_status_co...	old_ticket_amo...	new_ticket_amo...	action_ty...	changed_at
1	1	PAID	PAID	125.00	200.00	UPDATE	2026-05-01 15:19:49
2	1	PAID	PAID	200.00	200.00	UPDATE	2026-05-01 15:21:25

Ticket_Audit table recording changes after ticket updates

Triggers

485	-- =====
486	-- SECTION 15: TRIGGER FOR VIOLATIONS (UPDATE)
487	-- =====
488	DELIMITER \$\$
489	
490	• CREATE TRIGGER trg_violation_update
491	AFTER UPDATE ON Violations
492	FOR EACH ROW
493	BEGIN
494	INSERT INTO Violation_Audit
495	(
496	violation_id,
497	old_violation_type,
498	new_violation_type,
499	old_violation_location,
500	new_violation_location,
501	action_type
502	)
503	VALUES
504	(
505	OLD.violation_id,
506	OLD.violation_type,
507	NEW.violation_type,
508	OLD.violation_location,
509	NEW.violation_location,
510	'UPDATE'
511	);
512	END\$\$
513	
514	DELIMITER ;
515	

Trigger to capture updates to the Violations table

audit_id	violation_id	old_violation_ty...	new_violation_ty...	old_violation_locat...	new_violation_locati...	action_ty...	changed_at
1	1	Speeding	Reckless Driving	Grand Rapids, MI	Grand Rapids, MI	UPDATE	2026-05-01 15:23:29
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Violation_Audit table recording changes after violation updates

Two triggers were created to automatically track updates made to the Tickets and Violations tables. Each trigger captures both the original and updated values whenever a record is modified and inserts this information into the corresponding audit table. This includes key fields such as ticket status, ticket amount, violation type, and location, along with the action performed and a timestamp. These triggers ensure that all changes are logged without requiring manual input, supporting data accuracy and accountability within the system.

Validation Functions

<pre> -- This function checks whether a violator exists -- it returns 1 if the violator exists and 0 if not. ----- DELIMITER \$\$ CREATE FUNCTION fn_violator_exists(p_violator_id INT) RETURNS BOOLEAN READS SQL DATA > BEGIN DECLARE v_count INT; SELECT COUNT(*) INTO v_count FROM Violators WHERE violator_id = p_violator_id; RETURN v_count > 0; < END\$\$ DELIMITER ; </pre>	<pre> -- This function checks whether a ticket status code exists -- it returns 1 if the status exists and 0 if not. ----- DELIMITER \$\$ CREATE FUNCTION fn_ticket_status_exists(p_ticket_status_code VARCHAR(10)) RETURNS BOOLEAN READS SQL DATA ⊖ BEGIN DECLARE v_count INT; SELECT COUNT(*) INTO v_count FROM Ref_Ticket_Status WHERE ticket_status_code = p_ticket_status_code; RETURN v_count > 0; < END\$\$ DELIMITER ; </pre>
User-defined functions for validating violator and ticket status records	

Two validation functions were created to ensure data integrity within the database. The first function verifies that a given violator ID exists, while the second confirms that a valid ticket status code is provided. These functions are used to prevent invalid data from being processed during database operations.

Function Testing and Output

<pre> -- Should return 1 because violator_id 1 exists. SELECT fn_violator_exists(1) AS violator_exists_test; </pre>	<pre> violator_exists_t... 1 </pre>
<pre> -- Should return 0 because violator_id 999 does not exist. SELECT fn_violator_exists(999) AS violator_missing_test; </pre>	<pre> violator_missing_t... 0 </pre>
<pre> -- Should return 1 because PAID exists in Ref_Ticket_Status. SELECT fn_ticket_status_exists('PAID') AS status_exists_test; </pre>	<pre> status_exists_test 1 </pre>
<pre> -- Should return 0 because FAKE is not a valid status. SELECT fn_ticket_status_exists('FAKE') AS status_missing_test; </pre>	<pre> status_missing_test 0 </pre>

Test queries were executed to verify the functionality of each validation function. The results confirm that the functions correctly return true for valid inputs and false for invalid or non-existing values.

Stored Procedures (1)

<pre> DELIMITER \$\$ CREATE PROCEDURE sp_insert_ticket (IN p_violation_id INT, IN p_ticket_status_code VARCHAR(10), IN p_ticket_issue_date DATE, IN p_ticket_due_date DATE, IN p_ticket_paid_date DATE, IN p_ticket_amount DECIMAL(8,2), IN p_ticket_notes VARCHAR(255)) BEGIN -- Validate ticket status IF fn_ticket_status_exists(p_ticket_status_code) = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid ticket status code'; END IF; -- Insert if valid INSERT INTO Tickets (violation_id, ticket_status_code, ticket_issue_date, ticket_due_date, ticket_paid_date, ticket_amount, ticket_notes) VALUES (p_violation_id, p_ticket_status_code, p_ticket_issue_date, p_ticket_due_date, p_ticket_paid_date, p_ticket_amount, p_ticket_notes); END\$\$ DELIMITER ; </pre>	<pre> VALUES (p_violation_id, p_ticket_status_code, p_ticket_issue_date, p_ticket_due_date, p_ticket_paid_date, p_ticket_amount, p_ticket_notes); END\$\$ DELIMITER ; </pre>
Stored procedure for inserting ticket records with validation logic (formatted for readability)	
<pre> -- Valid insert CALL sp_insert_ticket(1, 'PAID', '2023-07-01', '2023-08-01', '2023-07-15', 120.00, 'Inserted via procedure'); </pre>	<pre> -- Invalid test (should ERROR) CALL sp_insert_ticket(1, 'FAKE', '2023-07-01', '2023-08-01', NULL, 120.00, 'This should fail'); </pre>
✓ 81 18:08:20 CALL sp_insert_ticket(1, 'PAID', '2023-07-01', '2023-08-01', '2023-07-15', 120.00, 'Inserted via procedure')	
✗ 82 18:10:43 CALL sp_insert_ticket(1, 'FAKE', '2023-07-01', '2023-08-01', NULL, 120.00, 'This should fail')	
Stored procedure test results showing successful insert and validation error handling	

This stored procedure is used to insert new ticket records into the database while enforcing validation rules. Before inserting the data, it checks whether the provided ticket status code is valid using a validation function. If the value is invalid, the procedure generates an error and

prevents the insert. The test results demonstrate both a successful insert with valid data and an error when invalid input is provided, confirming that the validation logic is working as expected.

Stored Procedures (2)

```

DELIMITER $$

CREATE PROCEDURE sp_update_ticket_status (
    IN p_ticket_number INT,
    IN p_new_ticket_status_code VARCHAR(10),
    IN p_ticket_paid_date DATE
)
BEGIN
    -- Validate ticket status before updating the ticket.
    IF fn_ticket_status_exists(p_new_ticket_status_code) = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Invalid ticket status code';
    END IF;

    -- Update the ticket if the status code is valid.
    UPDATE Tickets
    SET
        ticket_status_code = p_new_ticket_status_code,
        ticket_paid_date = p_ticket_paid_date
    WHERE ticket_number = p_ticket_number;
END$$

DELIMITER ;

```

Stored procedure for updating ticket status with validation logic

```

-- Valid update
CALL sp_update_ticket_status(2, 'PAID', '2023-07-01');

-- Check result
SELECT * FROM Tickets WHERE ticket_number = 2;

```

ticket_num...	violation_id	ticket_status_co...	ticket_issue_d...	ticket_due_da...	ticket_paid_d...	ticket_amou...	ticket_notes
2	2	PAID	2023-06-02	2023-07-02	2023-07-01	150.00	Awaiting payment

Successful execution of stored procedure updating ticket status

```

-- Invalid update test: this should fail
CALL sp_update_ticket_status(3, 'FAKE', NULL);

```

84 18:22:46 CALL sp_update_ticket_status(3, 'FAKE', NULL)

Stored procedure validation preventing an update with invalid ticket status

This stored procedure updates the status of an existing ticket while enforcing validation rules. Before performing the update, it checks whether the provided ticket status code is valid using a validation function. If the value is valid, the ticket is successfully updated; otherwise, the procedure generates an error and prevents the change. The test results demonstrate both a successful update and a failed attempt, confirming that the validation and error handling are functioning correctly.

Stored Procedures (3)

<pre> DELIMITER \$\$ • CREATE PROCEDURE sp_insert_violation (IN p_violator_id INT, IN p_vehicle_license_number VARCHAR(20), IN p_officer_badge_number VARCHAR(20), IN p_violation_datetime DATETIME, IN p_violation_location VARCHAR(150), IN p_violation_type VARCHAR(75), IN p_violation_description VARCHAR(255)) BEGIN -- Validate violator exists IF fn_violator_exists(p_violator_id) = 0 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid violator ID'; END IF; </pre>	<pre> -- Insert if valid INSERT INTO Violations (violator_id, vehicle_license_number, officer_badge_number, violation_datetime, violation_location, violation_type, violation_description) VALUES (p_violator_id, p_vehicle_license_number, p_officer_badge_number, p_violation_datetime, p_violation_location, p_violation_type, p_violation_description); END\$\$ DELIMITER ; </pre>
<i>Stored procedure for inserting violation records with validation logic</i>	
<pre> • -- Valid insert CALL sp_insert_violation(1, 'ABC1234', 'B1001', '2023-07-20 10:00:00', 'Grand Rapids, MI', 'Speeding', 'Inserted via procedure'); </pre>	<pre> -- Invalid test (should ERROR) • CALL sp_insert_violation(999, 'ABC1234', 'B1001', '2023-07-20 10:00:00', 'Grand Rapids, MI', 'Speeding', 'This should fail'); </pre>
<pre> 85 18:58:34 CALL sp_insert_violation(1, 'ABC1234', 'B1001', '2023-07-20 10:00:00', 'Grand Rapids, MI', 'Speeding', 'Insert... 1 row(s) affected </pre>	
<pre> 86 18:59:23 CALL sp_insert_violation(999, 'ABC1234', 'B1001', '2023-07-20 10:00:00', 'Grand Rapids, MI', 'Speeding', 'Thi... Error Code: 1644. Invalid violator ID </pre>	
<i>Stored procedure test results showing successful insert and validation error handling for violation records.</i>	

This stored procedure inserts new violation records into the database while enforcing validation rules. Before inserting the data, it checks whether the provided violator ID exists using a validation function. If the input is valid, the record is successfully inserted; otherwise, the procedure generates an error and prevents the insert. The test results demonstrate both a successful insert and a failed attempt, confirming that the validation logic is functioning correctly.

Reporting

SQL queries were developed to generate meaningful reports from the database, demonstrating how the stored data can be used for analysis and decision-making. These reports utilize joins to combine data across multiple related tables and apply aggregation techniques to summarize key information. The reporting section includes both a detailed report, which provides a comprehensive view of individual records, and a summary report, which highlights overall trends and patterns within the data.

Detailed Report

```
SELECT
    Tickets.ticket_number,
    Violators.violator_first_name,
    Violators.violator_last_name,
    Vehicles.vehicle_license_number,
    Vehicles.vehicle_make,
    Vehicles.vehicle_model,
    Violations.violation_type,
    Violations.violation_location,
    Officers.officer_badge_number,
    Tickets.ticket_amount,
    Tickets.ticket_status_code
FROM Tickets
JOIN Violations
    ON Tickets.violation_id = Violations.violation_id
JOIN Violators
    ON Violations.violator_id = Violators.violator_id
JOIN Vehicles
    ON Violations.vehicle_license_number = Vehicles.vehicle_license_number
JOIN Officers
    ON Violations.officer_badge_number = Officers.officer_badge_number;
```

Detailed report combining ticket, violation, vehicle, violator, and officer data using JOIN

ticket_num...	violator_fir...	violator_la...	vehicle_licen...	vehicle_make	vehicle_model	violation_type	violation_locati...	officer_badge_num...	ticket_amou...	ticket_status_co...
1	John	Smith	ABC1234	Toyota	Camry	Reckless Driving	Grand Rapids, MI	B1001	200.00	PAID
2	Emily	Johnson	XYZ5678	Honda	CR-V	Red Light	Wyoming, MI	B1002	150.00	PAID
3	Michael	Williams	LMN8910	Ford	F-150	Illegal Turn	Kentwood, MI	B1003	100.00	PAID
4	Sarah	Brown	QRS2345	Dodge	Caravan	Speeding	Grandville, MI	B1004	125.00	OVERDUE
5	David	Jones	TUV6789	Yamaha	MT-07	No Helmet	Walker, MI	B1005	75.00	ISSUED
6	Jessica	Garcia	JKL3456	BMW	M4	Expired Plates	Holland, MI	B1006	90.00	PAID
7	Daniel	Martinez	DEF7890	Chevrolet	Camaro	Speeding	Zeeland, MI	B1007	175.00	APPEAL
8	Ashley	Rodriguez	GHI1122	Subaru	Outback	Seatbelt	Byron Center, MI	B1008	65.00	PAID
9	Matthew	Davis	JKM3344	Freightliner	Cascadia	Overweight Load	Grand Rapids, MI	B1009	250.00	OVERDUE
10	Amanda	Lopez	NOP5566	Utility	Trailer	Improper Load	Wyoming, MI	B1010	200.00	ISSUED
11	Joshua	Gonzalez	RST7788	Toyota	Prius	Reckless Driving	Kentwood, MI	B1011	300.00	CANCELLED
12	Stephanie	Wilson	UVW9900	Tesla	Model 3	Speeding	Grandville, MI	B1012	125.00	PAID
13	Andrew	Anderson	AAA1111	Toyota	Corolla Hybrid	Distracted	Walker, MI	B1013	110.00	ISSUED
14	Nicole	Thomas	BBB2222	Jeep	Grand Chero...	Tailgating	Holland, MI	B1014	100.00	DISMISSED
15	Ryan	Taylor	CCC3333	Nissan	Altima	Speeding	Zeeland, MI	B1015	125.00	VOID
16	John	Smith	ABC1234	Toyota	Camry	Reckless Driving	Grand Rapids, MI	B1001	120.00	PAID
17	John	Smith	ABC1234	Toyota	Camry	Reckless Driving	Grand Rapids, MI	B1001	120.00	PAID

This detailed report combines data from multiple tables using JOIN operations to provide a comprehensive view of each traffic violation and its associated ticket. The report includes information about the violator, vehicle, violation details, issuing officer, and ticket status. This type of report is useful for reviewing individual cases and understanding how different parts of the database are connected.

Summary Report

<pre>-- =====: -- SECTION 21B: SUMMARY REPORT -- =====:</pre> <pre>SELECT ticket_status_code, COUNT(*) AS total_tickets, SUM(ticket_amount) AS total_amount FROM Tickets GROUP BY ticket_status_code;</pre>			
Summary report showing total tickets and total ticket amounts grouped by status			
	ticket_status_co...	total_tick...	total_amou...
	APPEAL	1	175.00
	CANCELLED	1	300.00
	DISMISSED	1	100.00
	ISSUED	3	385.00
	OVERDUE	2	375.00
	PAID	8	970.00
	VOID	1	125.00

This summary report groups ticket data by status to provide an overview of ticket activity within the system. It calculates the total number of tickets and the total dollar amount associated with each status category. This type of report is useful for identifying trends, such as how many tickets have been paid, are overdue, or are still pending.

Loom Link for the Recording:

<https://loom.com/share/f2b929ba1d214b4587e02f9bcb46dc3d>